AI-103: Developing AI Apps and Agents on Azure Master Cheat Sheet

Domain 1: Plan and manage an Azure AI solution (25–30%)

This domain covers planning, setting up, managing, and securing AI solutions on Azure, with a strong emphasis on responsible AI and agentic systems. It accounts for 25-30% of your score.

1.1 Choose the appropriate Foundry services for generative AI and agents

This section focuses on mapping business requirements to technical components: models, data retrieval, memory, and tools.

1.1.1 Choosing the Right Model Type

Foundry provides a unified model catalogue with 1,600+ models. Your choice depends on task complexity, latency, cost, and modality.

Model Type	Best For	Key Characteristics	Examples in Foundry
Large Language Models (LLMs)	Complex reasoning, summarization, conversation, code generation	High capability, higher latency/cost, general purpose	GPT-4o, GPT-4.1-mini, Claude 3.5 Sonnet, Llama 3
Small Language Models (SLMs)	Fine-tuned tasks, classification, entity extraction, high-volume simple tasks	Lower latency/cost, efficient, can be fine-tuned for specific domains	Phi-3 family, Mistral Small
Multimodal Models	Image understanding, video analysis, audio transcription, document layout analysis	Accepts text + images/audio/video as input; generates text	GPT-4o with vision, GPT-4o-mini, Jais, Reka Core (preview)

Model Type	Best For	Key Characteristics	Examples in Foundry
Foundry Tools	Extending agent capabilities (search, code execution, API calls)	Model-agnostic functions executed by the agent	Bing Search, Azure AI Search, Code Interpreter, MCP servers

Key Takeaway: Use the **Model Router** in Foundry to automatically direct queries to the optimal model based on your cost, latency, and accuracy requirements.

1.1.2 Choosing Foundry Services for Key Tasks

Task	Primary Service(s)	Purpose
Generative Tasks	Azure OpenAI Service, Model Catalog	Core text generation, summarization, translation.
Grounding	Azure AI Search	Connects LLM responses to your proprietary data to reduce hallucinations (Retrieval-Augmented Generation - RAG).
Vector Search	Azure AI Search (Vector Index)	Finds semantically similar content using embeddings; essential for RAG and memory retrieval .
Agent Workflows	Agent Service (LangGraph, Semantic Kernel)	Orchestrates multi-step plans, manages state, and coordinates multiple agents/tools .
Multimodal Processing	GPT-4o with vision, Azure AI Vision (in AI Services)	Analyzing images, videos, or document layouts within a workflow .

1.1.3 Choosing Methods for Retrieval and Indexing

- **Method Choice:**
    - **Keyword Search (BM25):** Use when exact term matching is critical (e.g., searching for specific error codes or part numbers).

- **Vector Search:** Use for semantic similarity (e.g., "Find policies about employee conduct" matching a procedure on professionalism). Requires **embeddings**.
- **Hybrid Search (Recommended):** Combines keyword and vector search results, re-ranked for maximum relevance.

- **Indexing Methods:**
  - **Push (SDK/REST API):** Your code ingests data into the index. Ideal for near real-time updates from event-driven architectures (e.g., Event Grid, Azure Functions).
  - **Pull (Indexers):** Azure AI Search pulls data from a source (Blob Storage, Cosmos DB, SQL Database) on a schedule. Best for periodic data refreshes from stable sources.

1.1.4 Choosing Memory, Tool, and Knowledge Integration

Agent systems require persistent context (memory) and the ability to act (tools).

Component	Purpose	Implementation in Foundry
Memory	Stores conversation history, user preferences, facts, and past actions across sessions. Types: Episodic (past events), Semantic (facts), Procedural (preferences) .	**Azure Cosmos DB** (with vector search for semantic memory), Redis Cache.
Tool Integration	Allows agents to perform actions (e.g., get weather, update database, send email).	**MCP (Model Context Protocol) Servers**, Function Calling, pre-built connectors (1,400+).
Knowledge Integration	Provides static or reference data for grounding.	**Azure AI Search** Knowledge Base, **Foundry Knowledge Bases** (unified RAG API) .

1.2 Set up AI Solutions in Foundry

This section covers the infrastructure and DevOps for deploying AI apps.

1.2.1 Designing Azure Infrastructure

- **Core Components:**

- **Azure AI Foundry Project:** The logical container for assets (models, data, agents, evaluations) .
- **Hub:** The parent resource that manages security, networking, and shared compute.
- **Azure AI Search:** Required for grounding and vector retrieval .
- **Storage Account:** Stores data, logs, and other artifacts.
- **Key Vault:** Manages secrets and keys .

- **Architecture Pattern:** Hub (centralized security/networking) + Project(s) (isolated dev/test/prod workspaces).

1.2.2 Choosing Deployment Options

Microsoft Foundry offers four primary deployment tiers to balance cost, latency, and throughput.

Offer Type	Best For	Key Features	Latency & Throughput
Standard (Pay-as-you-go)	Development, testing, variable production workloads	Global availability, default choice, no commitment required	Standard latency, shared throughput
Priority Processing	High-value, latency-sensitive production apps (real-time chat, decisioning)	Premium low-latency access, SLA-backed performance, same API as Standard	Low latency, higher throughput
Provisioned (PTU)	High-volume, predictable production workloads	Dedicated capacity (Provisioned Throughput Units), reserved for your workload	Predictable high throughput, consistent low latency
Batch	Large-scale offline processing (summarization,	Lower cost per token, optimized for asynchronous jobs	Lower throughput, higher latency

Offer Type	Best For	Key Features	Latency & Throughput
	evaluation runs, bulk generation)		(acceptable for offline)

Deployment Geography:

- **Global:** Best for latency and cost optimization with flexible data residency.
- **Data Zones (EU/US):** For data residency in a specific geography without single-region lock-in.
- **Regional:** For strict local compliance (e.g., Germany West Central).

We have 590 Practice test questions **AI-103: Developing AI Apps and Agents on Azure** (Taken from previous exams)

Full Practice Set link below

https://skillcertpro.com/product/microsoft-azure-ai-103-exam-questions-2026/

100% Money back Guarantee, If you don't pass the exam in 1st attempt, your money will be refunded back

1.2.3 Configuring Model and Agent Deployments

- **Models:** Deploy specific model versions (e.g., GPT-4, version 0613) with defined rate limits (tokens-per-minute, requests-per-minute). **Key setting:** Content Filter configuration (see 1.4.1).
- **Agents:** Deploy configured agents (with tools, instructions, and knowledge bases) as API endpoints. **Key settings:**
    - **Oversight Mode:** The degree of agent autonomy.
    - **Tool Access:** Which tools the agent can invoke.
    - **Memory Configuration:** How much history to retain.

1.2.4 Integrating CI/CD Pipelines

- **Goal:** Automate the deployment of models, agents, and evaluation pipelines from code to cloud.
- **Implementation:**
    1. **Source Control:** Store prompt templates, agent configuration (YAML/JSON), and evaluation code in GitHub.
    2. **Build Pipeline (GitHub Actions/Azure DevOps):** Lint code, run unit tests, and package assets.
    3. **Deploy Pipeline:**
        - Deploy infrastructure (Bicep/Terraform) for Foundry projects, AI Search, etc.
        - Register and deploy models using the Foundry SDK or CLI.
        - Deploy agents and configure their endpoints.
    4. **Gate/Approval:** After deployment, automatically run evaluation tests on the staging environment. Only pass to production if performance metrics (e.g., groundedness) pass thresholds.

1.3 Manage, Monitor, and Secure AI Systems

This section covers the operational lifecycle of an AI application, focusing on cost governance, performance tracking, data pipeline health, and zero-trust security models.

1.3.1 Manage quotas, scaling, rate limits, and cost footprints for model and agent workloads

Definition: Quotas are credit limits imposed by Azure to prevent budget overruns and manage capacity constraints. They define the maximum allowable throughput for your deployments.

Key Concepts:

- **Types of Quota:**
    - **Tokens Per Minute (TPM):** Used for **Standard (Pay-as-you-go)** deployments. Limits the volume of tokens processed.

  - **Provisioned Throughput Units (PTUs):** Used for **Provisioned** deployments. Represents dedicated hardware capacity reserved for your workload.
  - **Compute Cores:** Limits the number of virtual machine cores (CPU/GPU) available for compute instances and endpoints.

- **Shared Quota:** A temporary quota pool available in the Foundry portal for testing models (e.g., Llama, Phi, Mistral). Should **never be used for production** endpoints. Billing is usage-based.
- **Dynamic Spillover:** A feature where excess traffic spills over from PTU deployments to Standard (PAYG) quota to prevent HTTP 429 (Too Many Requests) errors.

Scaling & Cost Management:

- **Vertical Scaling:** Increasing PTUs or moving to a higher SKU (e.g., from Dv2 to NV series) to handle more load.
- **Horizontal Scaling:** Adding instances/replicas to distribute traffic.
- **Cost Reduction Strategies:**
  - **Provisioned (PTU):** Best for predictable, high-volume workloads (cost-efficient per token).
  - **Batch:** For asynchronous, high-volume offline processing (lowest cost) .
  - **Monitoring:** Use **Azure Cost Management** tags to track spending per project or team.

Practical Commands & Tips:

- **Viewing Quota:** In Foundry Portal -> **Management Center** -> **Quota**.
- **Requesting Increase:** Select the model deployment -> **Request quota**. Requires *Owner* or *Contributor* role at the subscription level.
- **Propagation Time:** Quota changes can take up to **15 minutes** to reflect.
- **KQL Query for Rate Limits:**

kusto

```
AzureDiagnostics
| where resultSignature_d == "429"
| summarize count() by OperationName, bin(TimeGenerated, 1h)
```

Key Takeaway: Match deployment type to workload pattern: **PTU for predictable production** (cost control), **Pay-as-you-go for variable/testing** (flexibility). Use **Shared Quota only for temporary tests**.

1.3.2 Monitor model performance, drift, safety events, and grounding quality

Definition: Model monitoring is the process of tracking the behavior of a model in production to detect degradation in performance, data integrity, or safety compliance compared to its baseline (training/validation data).

Key Metrics & Signals:

Monitoring Signal	Description	Key Metrics	Required Data
Data Drift	Changes in input data distribution over time.	Jensen-Shannon Distance, Population Stability Index (PSI), Wasserstein Distance.	Production inputs vs. Training data .
Prediction Drift	Changes in the model's output distribution.	PSI, Chebyshev Distance, Chi-Squared Test.	Production outputs vs. Validation data .
Data Quality	Checks for integrity issues in input data.	Null value rate, Type mismatch rate, Out-of-bounds rate.	Production inputs .
Feature Attribution Drift	Changes in which features are driving predictions.	Normalized Discounted Cumulative Gain (NDCG).	Production I/O vs. Training data .
Model Performance	Objective accuracy against real-world truth.	Classification: Accuracy, Precision, Recall. Regression: MAE, MSE, RMSE.	Production outputs vs. Ground truth data .

Monitoring Signal	Description	Key Metrics	Required Data
Safety Events	Blocked prompts or completions due to content filters.	Blocked request count, Harm severity (Low/Med/High).	Azure Monitor logs .
Grounding Quality	Alignment of LLM responses with retrieved documents.	Groundedness score (Prevents hallucinations).	Retrieved chunks vs. Model output.

Monitoring Setup:

1. **Enable Data Collection:** Turn on "Production inference data collection" for online endpoints.
2. **Configure Signals:** Set scheduled monitoring jobs (daily/weekly) based on data volume.
3. **Alerting:** Use **Azure Event Grid** to trigger retraining or notifications when drift exceeds thresholds.

Practical Commands & Tips:

- **Latency Metrics:** Track **Time to First Token (TTFT)** and **Time per Output Token (TPOT)** using resource logs.
- **Best Practice:** Start monitoring immediately after deployment to establish baseline. Use **training data** for data drift and **validation data** for prediction drift.

Key Takeaway: Drift is inevitable. Implement **Data Drift + Feature Attribution Drift** together to get early warnings of performance degradation. Use **Grounding metrics** specifically for RAG apps.

1.3.3 Monitor data ingestion quality, search index health, and relevance performance

Definition: Monitoring the vector search and indexing layer (Azure AI Search) to ensure data is being ingested correctly and retrieved efficiently.

Key Concepts:

- **Indexer Health:** Tracks the status of data pull jobs from sources (Blob, Cosmos DB, SQL).

- **Throttling (503 Errors):** Occurs when the search service capacity ("replicas" or "partitions") is exhausted, usually due to high QPM or indexing load .
- **Query Latency:** The time taken for the search service to return results (excluding network roundtrip).

Performance Analysis Tools:

1. **Metrics Charts (Azure Portal):** View *Search Latency*, *Queries Per Second (QPS)*, and *Throttled Queries %* on the Overview tab.
2. **Log Analytics (Kusto):** Write KQL queries against AzureDiagnostics to deep-dive.

Key Kusto Queries for Admins:

Use Case	KQL Snippet	Insight
Indexer Status	where OperationName == "Indexers.Status"	Checks if ingestion jobs are failing.
Long-running queries	where OperationName == "Query.Search"	sort by DurationMs to find slow queries.
Avg Query Latency	summarize avgif(DurationMs, ...)	Monitors overall search speed.
Throttling (503)	where resultSignature_d == "503"	Indicates capacity pressure.
Avg OPM (Indexing)	summarize countif(OperationName == "Indexing.Index")	Measures ingestion load.

Practical Tips:

- **Relevance Performance:** Use the **Search Explorer** in the portal to manually test queries and compare @search.score results.
- **Troubleshooting:** If latency spikes correlate with high "Indexing Operations Per Minute (OPM)", the indexer is starving queries. Scale up partitions or schedule indexing off-peak hours.

Key Takeaway: Separate monitoring for **data ingestion (Indexer)** vs. **data retrieval (Query)**. High **Throttling (503)** usually requires increasing **replicas**; high **latency** during ingestion requires increasing **partitions** or scheduling.

1.3.4 Configure security, including managed identity, private networking, keyless credentials, and role policies

Definition: Implementing identity-based access control (RBAC) and network isolation to secure Azure AI services without relying on hard-coded keys.

Key Configurations:

1. Keyless Credentials (Microsoft Entra ID)

- **Concept:** Code authenticates using an identity (managed identity or service principal) instead of an API key.
- **How to Implement:**
    - **Role Assignment:** Assign the Cognitive Services User role to the identity at the scope of the Azure AI Services resource.
    - **Code Change:** Use DefaultAzureCredential (Python/.NET) which automatically tries Managed Identity, Visual Studio, or CLI credentials.

```
python
from azure.ai.inference import ChatCompletionsClient
from azure.identity import DefaultAzureCredential
client = ChatCompletionsClient(endpoint="...", credential=DefaultAzureCredential())
```

2. Managed Identity (Two Types)

- **System-assigned:** Tied to a specific Foundry Hub/Project. Deleted when the resource is deleted.
- **User-assigned:** Standalone resource that can be assigned to multiple services (e.g., Foundry + VM).
- **Use Case:** Allows Foundry to access Azure AI Search or Storage without storing keys in environment variables.

3. Role Policies (Azure RBAC)

Role	Purpose	Typical Assignment
Cognitive Services Contributor	Admin access. Create deployments, manage models, configure content filters.	Platform Engineers.
Cognitive Services User	**Developer access only.** Call inference APIs (chat, completions). Cannot change configurations.	Application Developers, Agents .
Cognitive Services Usages Reader	Read Quota and Usage metrics.	FinOps / Monitoring teams.

4. Private Networking

- **Private Endpoint:** Assigns a private IP address to the AI Foundry resource from your Virtual Network (VNet). This **blocks all public internet access**.
- **VNet Integration:** Allows the Foundry resource to access data sources (e.g., Storage Accounts) sitting behind firewalls.

Best Practices (Exam Critical):

- **Production:** Do not rely on DefaultAzureCredential for production. Use a deterministic credential like ManagedIdentityCredential .
- **Key Revocation:** If keys were used previously, regenerate them (Portal -> Keys & Endpoints) after Entra ID is configured to enforce keyless auth .
- **Propagation:** Role assignments may take up to **5 minutes** to propagate.

Key Takeaway: Adopt **Keyless authentication** as the default. Remember the split: **Contributor** (Admin/Control Plane) vs. **User** (Data Plane/Inference). Use **Private Endpoints** to meet data residency/compliance requirements.

1.4 Implement Responsible AI Across Generative AI and Agentic Systems

1.4.1 Configuring Safety Filters and Guardrails

- **Safety Filters (Content Moderation):** Configure thresholds (Low, Medium, High) for four harm categories: Hate/Fairness, Sexual, Violence, Self-Harm.
    - **Application:** Applied to model input (user prompt) and output (model completion).
    - **Customization:** Can create custom lists of blocked terms (e.g., profanity, competitor names).
- **Guardrails (Foundry):**
    - **Prompt Shields:** Detect and block prompt injection attacks (e.g., "Ignore previous instructions").
    - **Protected Material Detection:** Identify potential copyright text in model outputs.
    - **Groundedness Detection:** Flag "unsubstantiated" claims (hallucinations) by comparing output to the provided source documents.
- **Risk Detection:** For Agents, monitor the "Plan of Action" for malicious intent before tool execution (e.g., deny a request to "delete all files").

1.4.2 Applying Responsible AI Instrumentation

- **Evaluators:** Reusable metrics to assess quality and safety.
    - **Performance:** Coherence, Fluency, Relevance.
    - **Safety:** Hate/Unfairness, Sexual, Violence, Self-Harm (GPT-based).
    - **Grounding:** Groundedness (checks response against retrieved documents).
- **Safety Evaluations (Fine-tuning):** Built-in safety evaluation on fine-tuning datasets and the resulting model. **Fails the job automatically** if harmful content detected.

- **Explanation Tooling:** Use **InterpretML** or model explainers to understand why a model made a specific classification or generated a token.

1.4.3 Implementing Auditing

- **Trace Logging (LangSmith, Foundry Traces):** Logs every step of an agent's execution: the user prompt, the model's internal reasoning (chain-of-thought), which tool was called, the tool's output, and the final response.
- **Provenance Metadata:** Includes the **citation** links to the source documents used for grounding in the response metadata.
- **Approval Workflows:** Integrate human review for high-stakes actions. The agent can pause execution and wait for an "approve" or "deny" signal from an external system (e.g., Logic App, custom API).

We have 590 Practice test questions **AI-103: Developing AI Apps and Agents on Azure** (Taken from previous exams)

Full Practice Set link below

https://skillcertpro.com/product/microsoft-azure-ai-103-exam-questions-2026/

100% Money back Guarantee, If you don't pass the exam in 1st attempt, your money will be refunded back

1.4.4 Governing Agent Behaviour

- **Oversight Modes:**
  - **Auto:** Agent executes all steps without intervention (for low-risk tasks).
  - **Semi-Auto:** Agent requires confirmation before executing certain tools (e.g., sending emails, making purchases).
  - **Manual:** Human-in-the-loop drives the action; agent only provides recommendations.
- **Constraints:** Define strict rules (e.g., "Never call the delete_user tool", "Never spend more than $50 per transaction").

- **Tool-Access Controls:** Implement **scoped credentials** (tokens with limited permissions). The "read" tool can have read-only access to the database, even if the agent is compromised.

Domain 2: Implement generative AI and agentic solutions (30–35%)

This domain covers the practical implementation of AI applications, covering 30-35% of the exam. It focuses on building generative AI apps (RAG), creating intelligent agents, and optimizing/operationalizing these systems in production.

2.1 Build Generative Applications by Using Foundry

This section covers creating LLM-powered applications, from simple model consumption to complex RAG pipelines.

2.1.1 Deploy and Consume LLMs, Small Models, Code Models, and Multimodal Models

Key Concepts:

- **Model Catalog:** The central repository in Azure AI Foundry where you discover, evaluate, and deploy models (1,600+ models including Azure OpenAI, Hugging Face, and Meta models) .
- **Deployment Process:**
    1. **Select Model:** Browse catalog → Filter by task, framework, or license.
    2. **Choose Deployment Type:** Standard (PAYG), Provisioned (PTU), or Batch.
    3. **Configure Endpoint:** Set rate limits (TPM), content filters, and authentication.
    4. **Deploy:** Creates an API endpoint with a unique URL and key/Entra ID access.

Model Types:

Model Type	Examples	Best For
Large Language Models (LLMs)	GPT-4o, GPT-4.1-mini, Llama 3	Complex reasoning, summarization, conversation, code generation
Small Language Models (SLMs)	Phi-3 family, Mistral Small	Fine-tuned tasks, classification, high-volume simple tasks
Code Models	CodeGemma, Code Llama	Code completion, generation, debugging
Multimodal Models	GPT-4o with vision	Image understanding, video analysis, document layout analysis

Consumption Methods:

- **REST API:** Standard HTTP requests to the endpoint.
- **Azure AI Inference SDK:** Python/NET SDKs with built-in retry and auth handling.
- **Playground:** Interactive testing in the Foundry portal before coding.

Key Takeaway: Match model complexity to task complexity. Use SLMs for simple, high-volume tasks to save costs. Deploy from the Model Catalog with a few clicks—no infrastructure coding required.

2.1.2 Implement Retrieval-Augmented Generation (RAG) in an Application

Definition: RAG combines LLMs with your proprietary data by retrieving relevant documents and injecting them as context, reducing hallucinations and grounding responses in facts.

RAG Architecture Flow:

text

User Query → Vector Search → Retrieved Context → LLM → Grounded Response

Implementation Steps:

1. **Ingestion:** Load documents into Azure AI Search.

2. **Chunking:** Split documents into smaller passages (e.g., 1024 tokens).
3. **Vectorization:** Convert chunks to embeddings using an embedding model (e.g., text-embedding-ada-002).
4. **Indexing:** Store vectors in an Azure AI Search vector index.
5. **Retrieval:** At query time, vector search finds semantically similar chunks.
6. **Generation:** Inject retrieved context into the LLM prompt and generate response.

Key Components:

- **Azure AI Search:** Primary vector store and search engine.
- **Embedding Model:** Converts text to vectors (provided by Azure OpenAI).
- **LLM:** Generates final grounded response (e.g., GPT-4o).

Key Takeaway: RAG is the primary pattern for grounding LLMs in your data. Always use hybrid search (vector + keyword) + semantic re-ranking for best relevance.

2.1.3 Design Workflows, Tool-Augmented Flows, and Multistep Reasoning Pipelines

Key Concepts:

- **Workflows:** Defined sequences of steps (retrieve → generate → validate).
- **Tool-Augmented Flows:** LLMs can call external tools (APIs, search, code interpreter) to accomplish tasks.
- **Multistep Reasoning:** Chain-of-thought where the model breaks complex problems into steps.

Design Patterns:

Pattern	Description	Use Case
Sequential	Step A → Step B → Step C	Simple, deterministic pipelines

Pattern	Description	Use Case
Conditional Branching	If condition, go to Step B, else Step C	Decision-based workflows
Tool-use Loop	LLM decides which tool to call, processes result, repeats	Dynamic problem-solving
Agentic	Autonomous planning and execution	Complex, multi-goal tasks

Implementation with Foundry:

- **Prompt Flow:** Visual designer for chaining LLM calls, tools, and logic.
- **LangGraph Integration:** Graph-based state machine for complex multi-step reasoning.

Key Takeaway: Start simple (sequential), add branching as needed, and use agentic patterns only for complex, dynamic tasks.

2.1.4 Evaluate Models and Apps, Including Detecting Fabrications, Relevance, Quality, and Safety

Definition: Systematic assessment of LLM outputs using automated evaluators (LLM-as-a-judge) to measure metrics like groundedness, relevance, and completeness.

RAG Evaluation Metrics (The "RAG Triad"):

Metric	What It Measures	Why It Matters
Retrieval	Are the retrieved documents relevant to the query?	Garbage in, garbage out. Poor retrieval = poor response.
Groundedness	Is the response supported by the retrieved context (no hallucinations)?	Prevents fabrication; ensures factual accuracy.
Relevance	Does the response fully address the user's question?	Answers correctly vs. just being truthful.

Additional Evaluators:

- **Response Completeness:** Measures recall against ground truth.
- **Document Retrieval (with qrels):** Calculates NDCG, XDCG, Fidelity scores for search quality.
- **Safety:** Detects hate speech, violence, self-harm, sexual content.

Evaluation Modes:

- **Local:** Run evaluators in-process (e.g., during development).
- **Cloud:** Submit batch evaluation jobs to Azure AI Foundry (scalable).

Practical Example (YAML config):

```
yaml
project_name: my-rag-app
mode: cloud
cloud_mode:
  evaluators:
    - groundedness
    - relevance
  thresholds:
    groundedness: 0.80
    relevance: 0.85
```

CI/CD Integration: Exit code 0 = all thresholds pass (merge allowed). Exit code 3 = threshold breach (block merge) .

Key Takeaway: Always evaluate RAG systems with the Triad (Retrieval, Groundedness, Relevance). Use threshold-based gates in CI/CD to prevent performance regressions.

2.1.5 Integrate Generative Workflows into Applications by Using Foundry SDKs and Connectors

Definition

Integrating generative workflows means embedding AI capabilities (models, agents, RAG pipelines) into your existing applications using Microsoft Foundry's programmatic interfaces rather than manually invoking APIs.

Key Concepts

What is a Hosted Workflow?

A workflow captures multi-step, multi-agent orchestration logic (like a code review loop or customer support escalation) and deploys it directly to Azure AI Foundry. This allows the workflow to be versioned, managed, and executed natively within Azure, unlike local workflows.

Integration Patterns

Pattern	Description	Best For
Connector-based	Pre-built connectors for Power Platform, Logic Apps, and third-party platforms	Low-code/no-code scenarios; rapid prototyping
Direct REST API	Direct HTTP calls to Foundry endpoints	Full control; custom applications
Foundry SDK	Azure AI Projects client library for Python/.NET	Native code integration; agent management

Using the Foundry SDK (Azure AI Projects Library)

The primary tool for code-first integration is the azure-ai-projects SDK.

Prerequisites:

bash

Install SDK

pip install --pre azure-ai-projects

Authenticate locally

az login

Core SDK Capabilities:

- **Agent Management**: Create, version, and delete agents

- **Workflow Registration**: Deploy YAML-defined orchestrations
- **Tool Integration**: Connect Azure AI Search, Functions, Code Interpreter, MCP servers
- **Client Creation**: Get OpenAI-compatible clients for model inference

Key SDK Operations

1. Client Initialization

python

```
import os
from azure.ai.projects import AIProjectClient
from azure.identity import DefaultAzureCredential

project_client = AIProjectClient(
    endpoint=os.environ["AZURE_AI_PROJECT_ENDPOINT"],
    credential=DefaultAzureCredential()  # Keyless authentication
)
```

<svg xmlns="http://www.w3.org/2000/svg" width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round" style="display:inline; vertical-align:middle; margin-right:4px;"> <path d="M12 8v4l3 3M12 22c5.523 0 10-4.477 10-10S17.523 2 12 2 2 6.477 2 12s4.477 10 10 10z"/> </svg> Exam Tip: DefaultAzureCredential tries multiple authentication methods sequentially (CLI → Managed Identity → VS Code). For production, use ManagedIdentityCredential directly.

2. Registering a Workflow Agent

python

```
from azure.ai.projects.models import WorkflowAgentDefinition

workflow_agent = project_client.agents.create_version(
    agent_name="CodeReviewWorkflow",
    definition=WorkflowAgentDefinition(workflow=yaml_definition),
    description="Multi-agent code review loop"
)
```

Output: Created workflow: CodeReviewWorkflow:1

3. Using Connections to Access Resources

python

```
# List all connected resources in the project
for connection in project_client.connections.list():
    print(f"{connection.name} ({connection.type})")
```

Connections include Azure AI Search, Storage, Azure OpenAI, etc.

4. Performing Agent Operations with OpenAI Client

python

```
with project_client.get_openai_client() as openai_client:
    # Create agent version
    agent = project_client.agents.create_version(
        agent_name="SupportAgent",
        definition=PromptAgentDefinition(
            model="gpt-4.1-mini",
            instructions="You are a helpful support assistant"
        ),
    )

    # Run conversation
    response = openai_client.responses.create(
        model="gpt-4.1-mini",
        input="What is the refund policy?",
    )
    print(response.output_text)
```

The get_openai_client() method returns an authenticated client compatible with the OpenAI Responses API.

Workflow Definition (YAML)

Hosted workflows are defined declaratively in YAML:

yaml

```
kind: workflow
trigger:
  kind: OnConversationStart
  actions:
    - kind: SetVariable
      variable: Local.UserQuery
      value: =UserMessage(System.LastMessageText)

steps:
  - kind: InvokeAzureAgent
    agent: DeveloperAgent
    input: =Local.UserQuery
    output: Local.CodeOutput

  - kind: ConditionGroup
    conditions:
      - condition: =Contains(Local.CodeOutput, "approved")
        actions:
          - kind: EndConversation
    elseActions:
      - kind: GotoAction
        actionId: invoke_developer  # Loop back
```

Comparison: SDK vs REST API vs Connectors

Aspect	Foundry SDK	Direct REST API	Connectors
Control	High	Full	Low
Setup Complexity	Medium	High	Low (click/configure)

Aspect	Foundry SDK	Direct REST API	Connectors
Best For	Custom .NET/Python apps	Any language; custom HTTP	Power Apps, Logic Apps, Power Automate
Authentication	Entra ID (using DefaultAzureCredential)	API keys or Entra ID tokens	Managed by platform

2.1.6 Configure an Application to Connect to a Foundry Project

Definition

Configuring a connection involves setting up your application with the correct endpoint, authentication method, and permissions to interact with Azure AI Foundry's data plane APIs.

Key Configuration Elements

Configuration Element	Description	Example
Project Endpoint	The unique URL of your Foundry project	https://<account>.services.ai.azure.com/api/projects/<project>
API Version	REST API version (for direct calls)	2025-01-01-preview
Authentication	Entra ID (only supported method)	DefaultAzureCredential or API keys (legacy)
Model Deployment Name	The name of your deployed model	gpt-4.1-mini

Configuration Element	Description	Example
Agent Name	For agent-based interactions	hosted-agent-readiness-coach

Connection Methods

Method 1: Environment Variables (Recommended)

bash

```
export AZURE_AI_PROJECT_ENDPOINT="https://myaccount.services.ai.azure.com/api/projects/myproject"
export AZURE_AI_MODEL_DEPLOYMENT_NAME="gpt-4.1-mini"
```

In code, the SDK reads these variables automatically.

Method 2: Configuration File (appsettings.json)

json

```
{
  "Foundry": {
    "ProjectEndpoint": "https://<account>.services.ai.azure.com/api/projects/<project>",
    "AgentName": "my-agent",
    "ApiVersion": "2025-01-01-preview"
  }
}
```

For .NET Blazor apps.

Method 3: Hardcoded (Not recommended for production)

python

```
project_client = AIProjectClient(
    endpoint="https://myaccount.services.ai.azure.com/api/projects/myproject",
    credential=DefaultAzureCredential()
)
```

Authentication Deep Dive

Entra ID (Microsoft Entra ID) is the ONLY supported authentication method for the SDK.

For Local Development (DefaultAzureCredential **sequence):**

1. Environment variables (client secret/certificate)
2. Workload Identity (Kubernetes)
3. **Managed Identity** (Azure-hosted)
4. **Azure CLI credentials** (az login – most common for dev)
5. Azure PowerShell
6. Visual Studio Code
7. Interactive browser (fallback)

For Production (Azure-hosted):

- Use **Managed Identity** (System-assigned or User-assigned)
- Assign the Azure AI User role to the identity

Verifying Authentication:

bash

Check if you can get a token

az account get-access-token –resource “https://ai.azure.com”

Ensure correct subscription

az account set –subscription “<subscription-id>”

Role-Based Access Control (RBAC)

Role	Permissions
Azure AI User	Send inference requests; interact with deployed models/agents
Azure AI Developer	Create and manage deployments, agents, evaluations

Role	Permissions
Azure AI Contributor	Full access to Foundry resources
Reader	View-only access

Exam Tip: End users only need the Azure AI User role to interact with an application that calls Foundry.

Connection Troubleshooting

Error	Likely Cause	Solution
401 Unauthorized	Authentication failed	Run az login again; verify tenant/subscription
404 Not Found	Wrong endpoint or agent name	Verify AZURE_AI_PROJECT_ENDPOINT and agent name
TokenCredential auth error	No token source available	Run az account get-access-token --resource "https://ai.azure.com"
Connection refused	Agent container stopped	Check agent status in Foundry portal
CORS error	Direct browser-to-API call	Route through backend server; don't call Foundry directly from browser JS

Complete Configuration Example

Goal: Build a chat UI that calls a hosted agent in Foundry.

Step 1: Set environment variables

bash

```
export AZURE_AI_PROJECT_ENDPOINT="https://myaccount.services.ai.azure.com/api/projects/myproject"
```

Step 2: Initialize client

python

```
from azure.ai.projects import AIProjectClient
from azure.identity import DefaultAzureCredential


project_client = AIProjectClient(
    endpoint=os.environ["AZURE_AI_PROJECT_ENDPOINT"],
    credential=DefaultAzureCredential()
)
```

Step 3: Get OpenAI client and run agent

python

```
with project_client.get_openai_client() as openai_client:
    response = openai_client.responses.create(
        model="gpt-4.1-mini",
        input="Hello, can you help me?",
        extra_body={"agent_reference": {"name": "SupportAgent", "type": "agent_reference"}}
    )
    print(response.output_text)
```

Foundry SDK Key Features Summary

Feature	Method	Preview Status
Agent operations	project_client.agents.*	✅ General availability
Connections	project_client.connections.list()	✅ General availability
Agent tools (Code Interpreter, File Search, etc.)	project_client.agents.* with tool objects	✅ General availability
Workflows (multi-agent orchestration)	WorkflowAgentDefinition	🟡 Preview

Feature	Method	Preview Status
Memory stores	project_client.beta.memory_stores	🟡 Preview
Evaluations	project_client.evaluation_rules	🟡 Preview

2.2 Build Agents by Using Foundry

This section covers creating intelligent agents that can plan, use tools, remember context, and execute multi-step tasks autonomously.

2.2.1 Define Agent Roles, Goals, Conversation Tracking, and Tool Schemas

Definition: An agent is an LLM-powered system that can observe its environment, reason about goals, and take actions using tools.

Agent Configuration Elements:

- **Role/Persona:** Defines the agent's identity and expertise (e.g., "You are a helpful customer support agent for Contoso Ltd.").
- **Goal:** The primary objective (e.g., "Resolve customer issues by searching knowledge base and creating tickets").
- **Conversation Tracking:** Memory strategy (short-term: session history; long-term: vector store of past interactions).
- **Tool Schemas:** JSON definitions of what tools exist, their parameters, and expected outputs.

Example Agent Definition:

```
json
{
 "name": "SupportAgent",
 "instructions": "You help customers with technical issues. Always search KB first.",
 "tools": ["search_kb", "create_ticket"],
```

```
 "memory": "conversation_buffer",
 "max_turns": 10
}
```

Key Takeaway: Clear, specific instructions produce better agent behavior. Define tools with precise schemas for reliable function calling.

We have 590 Practice test questions **AI-103: Developing AI Apps and Agents on Azure** (Taken from previous exams)

Full Practice Set link below

https://skillcertpro.com/product/microsoft-azure-ai-103-exam-questions-2026/

100% Money back Guarantee, If you don't pass the exam in 1st attempt, your money will be refunded back

2.2.2 Build Agents That Integrate Retrieval, Function-Calling, and Conversation Memory

Core Agent Capabilities:

- **Retrieval (RAG):** Agent can query a knowledge base to ground responses in facts .
- **Function-Calling:** Agent can invoke tools by matching user intent to function signatures.
- **Conversation Memory:** Maintains context across turns (short-term) and sessions (long-term).

Implementation Approaches:

1. **Prompt-Based Agent:** Define instructions + tools in system prompt. Best for simple agents.
2. **Hosted Agent (Container):** Custom logic + LLM orchestration. Best for complex workflows.
3. **Agent Framework (LangGraph/Semantic Kernel):** State machines with explicit planning steps.

Memory Types:

Memory Type	Storage	Use Case
Buffer	Session only	Short conversations
Vector Store	Persistent (Cosmos DB + vector search)	Long-term facts, user preferences
Summary	Session only	Long conversations (summarize history)

Key Takeaway: Start with prompt-based agents. Move to hosted/framework agents when you need complex state management or custom orchestration.

2.2.3 Integrate Agent Tools, Including APIs, Knowledge Stores, Search, and Custom Functions

Tool Categories:

- **APIs:** REST endpoints (e.g., weather API, internal CRM).
- **Knowledge Stores:** Vector databases, Azure AI Search indexes.
- **Search:** Bing Search, Azure AI Search web queries.
- **Content Understanding:** Document intelligence, computer vision.
- **Custom Functions:** Python/C# code running in containers.

Tool Registration Example:

```
python
@tool
def get_weather(city: str) -> str:
    """Get current weather for a city."""
    # API call logic
    return f"Weather in {city}: 72°F, sunny"
```

Best Practices:

- Provide clear, descriptive function names and parameter schemas.
- Handle errors gracefully (return user-friendly error messages).
- Implement idempotency for write operations.

Key Takeaway: Tools extend agent capabilities beyond text generation. Each tool needs a clear schema and error handling.

2.2.4 Implement Orchestrated Multi-Agent Solutions

Definition: Multiple specialized agents collaborating to solve complex tasks, coordinated by an orchestrator.

Architecture Patterns:

Pattern	Description	When to Use
Manager-Worker	One agent allocates tasks to specialized workers	Clear task decomposition
Peer-to-Peer	Agents communicate directly (handoffs)	Natural conversation flow
Hierarchical	Multiple levels of management	Very complex, enterprise-scale

Scaling Challenges:

- **Concurrency:** Multiple agents hit the same LLM deployment → queuing → latency.
- **Token Throughput:** Costs scale with number of agents × turns.
- **Timeout Risk:** Complex multi-agent chains can exceed orchestration timeouts (~10 minutes) .

Optimization Strategies:

- Use different models for different agents (GPT-4o-mini for simple tasks).
- Implement asynchronous processing (Service Bus + Azure Functions).
- Cache frequent responses (Redis).
- Use manager agent to route tasks efficiently.

Key Takeaway: Multi-agent systems increase capability but also complexity, latency, and cost. Start with single agent, scale to multiple only when necessary.

2.2.5 Build Autonomous or Semiautonomous Workflows with Safeguards and Approval Flows

Definition: Agents can operate with varying degrees of autonomy, from fully manual (human drives actions) to fully auto (agent executes without intervention) .

Oversight Modes:

Mode	Description	Tool Execution
Auto (Autonomous)	Agent executes all steps without human intervention	Immediate
Semi-Auto	Agent requires confirmation for certain actions	Approval required
Manual	Human-in-the-loop drives all actions; agent only recommends	None (human executes)

Safeguards:

- **Rate Limits:** Maximum tool calls per minute.
- **Budget Controls:** Maximum cost per task or session.
- **Tool Restrictions:** Deny list (e.g., "never call delete_user").
- **Timeouts:** Maximum execution time per agent turn (e.g., 10 minutes for high-reasoning tasks) .

Approval Flow Example:

1. Agent plans: "Search KB → Create ticket → Send email"
2. System pauses at "Create ticket" (high-risk action)
3. Approval request sent to manager via Teams/email
4. Upon approval, agent continues

Key Takeaway: Use semi-autonomous mode for production agents. Start with more oversight (manual/approval), then increase autonomy as confidence grows.

2.2.6 Integrate Monitoring into Deployed Agents, Evaluate Agent Behavior, and Perform Error Analysis

Monitoring Components:

- **Tracing:** Captures every LLM call, tool invocation, and decision step.
- **Evaluation:** Batch tests to measure agent quality over time.
- **Error Analysis:** Identifies failure patterns (e.g., tool mis-selection, timeout loops).

Tracing with Azure AI Foundry:

- Automatic tracing for Microsoft Agent Framework and LangGraph.
- Traces appear in portal under "Observability > Traces" within 2-5 minutes.
- Captures: request input, response output, tool calls, latency, token usage.

Evaluation Datasets:

- Harvest production traces → Create evaluation dataset.
- Version datasets to track performance over time.
- Detect regressions by comparing eval scores across versions.

Error Analysis Workflow:

1. Identify low-performing query in production trace.
2. Add to evaluation dataset.
3. Run batch evaluation on new agent version.
4. Verify fix before deploying.

Key Takeaway: Tracing is non-negotiable for production agents. Harvest production failures into evaluation datasets to prevent regression.

2.3 Optimize and Operationalize Generative AI Systems

This section covers fine-tuning, prompt optimization, observability, and hybrid orchestration for production AI systems.

2.3.1 Tune Generation Behavior, Such as Prompt Engineering and Adjusting Model Parameters

Key Parameters:

Parameter	Effect	Recommended Values
Temperature	Randomness (0 = deterministic, 1 = creative)	0.1-0.3 for factual tasks; 0.7-0.9 for creative
Top-P	Nucleus sampling (cumulative probability threshold)	0.9-0.95 typically
Max Tokens	Maximum length of response	Varies by task (e.g., 500 for Q&A)
Reasoning Effort	Thinking time (Minimal/Low/Medium/High). Higher = better quality, slower, more expensive	Medium for general; High only for complex planning

Prompt Engineering Techniques:

- **Zero-shot:** Direct instruction ("Summarize this document").
- **Few-shot:** Provide examples in prompt.
- **Chain-of-Thought:** "Let's think step by step."
- **Structured Outputs:** Constrain response format (JSON, XML) .

Key Takeaway: Start with temperature 0.2 for factual tasks. Use reasoning_effort = HIGH sparingly (only for the hardest steps) as it significantly increases latency.

2.3.2 Implement Model Reflection, Chain-of-Thought Evaluations, and Self-Critique Loops

Definition: Techniques where the model evaluates its own outputs to improve quality.

Self-Critique Patterns:

1. **Generate → Reflect → Revise:** Model generates answer, then critiques it, then revises.
2. **Multi-Persona:** Different "expert" roles discuss and refine.

3. **Chain-of-Thought Evaluation:** Model explains its reasoning behind evaluating an output.

Implementation Example:

Step 1: Generate initial answer to user query.

Step 2: "Critique your answer. Is it accurate? Does it fully answer the question?"

Step 3: "Based on your critique, provide an improved answer."

Benefits: Higher accuracy, better groundedness, reduced hallucinations. **Cost:** 2-3x more tokens.

Key Takeaway: Use self-critique for high-stakes responses (accuracy-critical). Not recommended for real-time/high-volume scenarios.

2.3.3 Set Up Observability by Implementing Tracing, Token Analytics, Safety Signals, and Latency Breakdowns

Observability Pillars (Azure AI Foundry):

Signal	What It Captures	Tool
Tracing	End-to-end execution: LLM calls, tool invocations, decisions	Azure AI Foundry Traces + App Insights
Token Analytics	Token consumption per request, per model, per user	Azure Monitor + Cost Management
Safety Signals	Content filter violations, prompt injection attempts	Content Safety logs
Latency Breakdowns	Time-to-first-token (TTFT), token generation time, tool execution time	Traces with span timing

Implementation Steps:

1. Instrument application with Azure AI Inference SDK (automatic tracing).
2. Configure integration with Application Insights.
3. Set up dashboards for key metrics (token usage, latency, error rate).

4. Configure alerts for anomalies (e.g., sudden latency spike).

We have 590 Practice test questions **AI-103: Developing AI Apps and Agents on Azure** (Taken from previous exams)

Full Practice Set link below

https://skillcertpro.com/product/microsoft-azure-ai-103-exam-questions-2026/

100% Money back Guarantee, If you don't pass the exam in 1st attempt, your money will be refunded back

KQL Query Example (Trace Analysis):

```
kusto
traces
| where message contains "agent_execution"
| extend duration = todouble(customDimensions.duration)
| summarize avg(duration), p95(duration) by operation_Name, bin(timestamp, 1h)
```

Key Takeaway: Tracing is your primary debugging tool for agent failures. Always instrument early, not after issues arise.

2.3.4 Orchestrate Multiple Models, Flows, or Hybrid LLM and Rules Engines

Definition: Routing requests across different models or combining LLMs with deterministic rule engines for optimal cost/quality/latency.

Orchestration Strategies:

Strategy	Description	When to Use
Model Router	Automatically route queries to cheapest model that can handle the task (e.g., simple Q&A → GPT-4o-mini; complex → GPT-4o)	Cost optimization
Semantic Router	Route based on query intent (e.g., "weather" → weather API; "talk" → LLM)	Mixed query types

Strategy	Description	When to Use
Conditional LLM/Rules	Use deterministic rules for simple cases, LLM for edge cases	Hybrid systems
LLM Caching	Cache frequent queries (Redis) to avoid duplicate LLM calls	High-volume repetitive workloads

Hybrid (LLM + Rules) Example:

text

User Query → Intent Classifier (LLM) → Route:

- "Check balance" → Rule engine (API call)

- "Explain policy" → LLM with retrieval

- "Complaint" → Escalate to human

Multi-Deployment Load Balancing:

- Deploy same model to multiple regions/endpoints.
- Use Azure Front Door or load balancer to distribute traffic.
- Prevents single-endpoint overload.

Key Takeaway: Not every request needs GPT-4o. Use model routers to reduce costs by 50-80% without quality degradation.

Domain 3: Implement computer vision solutions (10–15%)

This domain covers implementing computer vision solutions on Azure, focusing on both **generation** (creating images/videos) and **understanding** (analyzing visual content). It accounts for 10-15% of the exam. The key distinction is between using Azure OpenAI for generation and Azure Vision/Content Understanding for analysis.

3.1 Design and Implement Image- and Video-Generation Solutions

This section covers creating and editing visual content using AI models.

3.1.1 Implement a Solution That Generates Images from Text Prompts and Reference Media

Definition: Image generation is the process of creating new visual content from natural language descriptions (text-to-image) or by modifying reference images.

Primary Azure Service:

- **Azure OpenAI with DALL-E models** – Microsoft's primary offering for image generation from natural language
- **Microsoft MAI-Image-2** – Microsoft's in-house image generation model, available in Foundry

Key Concepts:

Concept	Description
Text-to-Image	Generate entirely new images from descriptive prompts
Image-to-Image	Modify or transform existing reference images using prompts
Prompt Engineering	Crafting detailed, specific prompts for better results (e.g., "photorealistic," "cinematic lighting")
Resolution Control	Setting output dimensions (e.g., 1024x1024, 1792x1024)

Implementation Steps:

1. Deploy an image generation model (DALL-E or MAI-Image-2) in Foundry
2. Use the Azure OpenAI REST API or SDK with image generation endpoints
3. Craft prompts with specific details (style, composition, lighting, colors)
4. Handle the response (image URL or base64-encoded image)

Example API Call (Conceptual):

```
python
response = openai.Image.create(
  model="dall-e-3",
  prompt="A serene mountain landscape at sunset, photorealistic style",
  n=1,
  size="1024x1024"
)
image_url = response.data[0].url
```

When to Use vs. Not Use:

Use Azure OpenAI for Image Generation	Don't Use for
Generating images from natural language	Specific domain extraction (use Document Intelligence)
Creative/marketing asset creation	Structured field extraction (use Content Understanding)
Rapid prototyping of visuals	Open source model hosting (use Azure ML)

3.1.2 Implement a Solution That Generates Videos from Text Prompts and Reference Media

Definition: Video generation creates new video content from text descriptions or by transforming existing media.

Current State (2026):

- Microsoft MAI-Image-2 supports both **image and video generation** capabilities
- Video generation is an emerging capability; expect integration through Foundry
- Pricing: Approximately $33 per 1M output tokens for generated images (video pricing similar structure)

Key Concepts:

- **Text-to-Video:** Generate short video clips from descriptive prompts
- **Frame Consistency:** Ensuring smooth transitions between generated frames
- **Temporal Coherence:** Maintaining object identity across time

Practical Tip: For the exam, focus on understanding that video generation is available through MAI-Image-2 in Foundry and that it follows similar patterns to image generation but with temporal considerations.

We have 590 Practice test questions **AI-103: Developing AI Apps and Agents on Azure** (Taken from previous exams)

Full Practice Set link below

https://skillcertpro.com/product/microsoft-azure-ai-103-exam-questions-2026/

100% Money back Guarantee, If you don't pass the exam in 1st attempt, your money will be refunded back

3.1.3 Configure Image-Editing Workflows, Including Inpainting, Mask-Based Edits, and Prompt-Driven Modifications

Definition: Image editing workflows allow AI to modify specific regions of an image while preserving the rest.

Key Editing Techniques:

Technique	Description	Use Case
Inpainting	Fill in missing or damaged parts of an image	Remove unwanted objects, restore old photos
Mask-Based Edits	Apply changes only to masked (selected) regions using a mask image	Selective color changes, object replacement
Prompt-Driven Modifications	Describe the change you want; AI identifies and applies it	"Make the sky more dramatic," "Change the car color to red"

How Inpainting Works:

1. User provides original image + mask (white = area to edit, black = keep unchanged)
2. User provides text prompt describing desired content for masked area
3. Model generates content only within masked region
4. Result blends seamlessly with surrounding image

Implementation Considerations:

- Masks can be user-drawn or AI-generated (e.g., segment anything models)
- Prompt specificity affects quality ("a red car" vs. "a vintage red Ferrari sports car")
- Available through DALL-E and MAI-Image-2 in Foundry

3.1.4 Implement Workflows to Edit Generated Videos

Definition: Applying AI-powered edits to existing video content, including trimming, style transfer, object removal, or recutting based on prompts.

Key Capabilities:

- **Temporal Editing:** Apply changes consistently across video frames
- **Object Removal:** Remove specified objects throughout a video segment
- **Style Transfer:** Apply artistic styles to entire video
- **Scene Detection:** Automatically identify scene boundaries for targeted edits

Current State: Video editing capabilities are less mature than image editing. For the exam, understand that:

- Video editing extends the same principles as image editing (inpainting, masks, prompts) but applied temporally
- Integration is through Foundry tools and Azure OpenAI's multimodal capabilities

3.1.5 Select and Apply Appropriate Generation and Editing Controls

Definition: The various parameters and controls available for fine-tuning generation and editing outputs.

Key Controls:

Control	Parameter	Effect
Quality/Resolution	quality: standard/hd	Higher quality = more detail, longer generation time
Size	size: 1024x1024, 1792x1024	Output dimensions affect aspect ratio and detail
Style	style: natural/vivid	Natural = realistic; Vivid = more dramatic/saturated
Number of Images	n: 1-10	How many variations to generate per prompt
Prompt Strength	(Prompt guidance scale)	Higher = follows prompt more strictly; Lower = more creative
Editing Mode	inpainting, editing	Determines whether to regenerate entire image or modify masked region

Selection Criteria:

Scenario	Recommended Control Settings
High-quality marketing asset	Quality: HD, Size: large (1792x1024), Style: natural
Rapid prototyping	Quality: standard, n: 3-4 for variations
Product image editing	Inpainting with precise mask
Social media content	Standard quality, square (1024x1024)

3.2 Design and Implement Multimodal Understanding Workflows

This section covers analyzing and extracting information from images, video, and audio using AI services.

3.2.1 Build a Solution That Analyzes Visual Context by Using Multimodal Models

Definition: Multimodal models (like GPT-4o with vision) can analyze images, video frames, and audio alongside text to understand visual context.

Primary Service:

- **Azure OpenAI GPT-4o with vision** – Multimodal model for broad visual analysis
- **Azure Content Understanding** – For structured extraction from visual content

When to Use Each:

Use Azure OpenAI (GPT-4o with vision)	Use Azure Content Understanding
Broad, non-specific image analysis	Structured field extraction with defined schemas
General understanding of visual content	Domain-specific analyzers (invoices, receipts, IDs)
Conversational Q&A about images	RAG-optimized extraction (markdown, tables)
Quick caption/description generation	Production-scale extraction workflows

Implementation Example:

python

```
# Conceptual - GPT-4o with vision API
response = openai.ChatCompletion.create(
  model="gpt-4o",
  messages=[
    {
      "role": "user",
      "content": [
        {"type": "text", "text": "What is happening in this image?"},
        {"type": "image_url", "image_url": {"url": image_url}}
      ]
```

```
    }
  ]
)
```

3.2.2 Configure Apps to Produce Concise or Detailed Captions for Single or Multiple Images

Definition: Automatically generating textual descriptions of image content, with configurable verbosity levels.

Azure Services for Captioning:

- **Azure Vision Image Analysis** – Provides image captioning as a built-in feature
- **Azure OpenAI GPT-4o with vision** – For customizable, prompt-controlled captioning

Caption Types:

Type	Description	Length	Use Case
Concise	Single sentence, key objects and actions	~10-20 words	Quick understanding, search indexing
Detailed	Multiple sentences, relationships, context	~50-100 words	Accessibility, detailed documentation
Multi-image	Comparative or sequential descriptions across images	Varies	Storyboard analysis, slide decks

Configuration Tips:

- Use prompt engineering to control caption length: "Describe this image in one short sentence" vs. "Provide a detailed description"
- Azure Vision API allows configuring caption confidence thresholds

Accessibility Note: Captions are critical for alt-text generation (see 3.2.4). Microsoft products like PowerPoint, Word, and Edge use this capability.

3.2.3 Implement a Solution That Enables Question-Answering Grounded in Visual Evidence

Definition: Enabling users to ask questions about visual content and receive answers supported by specific visual evidence (e.g., "What color is the car?" → "The car is red").

Key Capabilities:

- **Visual Question Answering (VQA):** Model answers questions based strictly on image content
- **Grounding:** Responses can reference specific regions of the image
- **Multimodal Reasoning:** Combining visual and textual information

Architecture Approaches:

Approach	Description	Best For
GPT-4o with vision (direct)	Single model that sees and answers	General VQA, conversational interfaces
RAG with visual indexing	Extract visual features → vector search → LLM	Large collections of images, document images with text
Azure Content Understanding	Custom analyzers for specific question types	Domain-specific VQA (invoices, forms)

Implementation Example (Conceptual):

User: "What documents are shown in this image?",

Image ID: "img_12345",

Response: "The image shows a passport (bottom-left) and a driver's license (right)."

3.2.4 Configure Generation of Alt-Text and Extended Image Descriptions Aligned to Accessibility Guidelines

Definition: Generating alternative text (alt-text) for images that meets Web Content Accessibility Guidelines (WCAG) standards.

Why It Matters:

- Legal compliance (ADA, Section 508, WCAG 2.1)

- Screen reader compatibility for visually impaired users
- Improved SEO

Alt-Text Guidelines:

Type	Description	Length	WCAG Requirement
Standard alt-text	Brief description of image purpose	1-2 sentences (125 chars max recommended)	1.1.1 (Non-text Content)
Extended description	Detailed explanation for complex images	Multiple sentences, can be hidden	Can use longdesc or adjacent text

Azure Implementation:

- Use **Azure Vision Image Analysis** captioning models
- Configure for concise output for standard alt-text
- For extended descriptions, use GPT-4o with vision with prompt: "Provide a detailed WCAG-compliant image description"

Configuration Example:

```
python
# Request alt-text appropriate description
response = vision_client.describe_image(
    image_url,
    max_candidates=1,
    # For alt-text: concise, objective, descriptive
    language="en",
    model_version="latest"
)
alt_text = response.captions[0].text
```

3.2.5 Implement Visual Understanding by Configuring Azure Content Understanding in Foundry Tools to Extract Visual Characteristics

Definition: Azure Content Understanding is a multimodal AI service that extracts semantic content from documents, video, audio, and images, transforming unstructured content into structured data.

Key Analyzer Types:

Analyzer	Input Type	Output
prebuilt-documentSearch	PDF, Office docs, images with text	Markdown, tables, figures, layout, one-paragraph summary
prebuilt-imageSearch	Standalone images	One-paragraph description optimized for search
prebuilt-videoSearch	Video files	Visual frames, audio transcription, structured summaries with temporal alignment
prebuilt-audioSearch	Audio files	Transcripts, speaker diarization, timing, summaries

Custom Analyzers:

- Build domain-specific analyzers for specialized extraction needs
- Support all four modalities: documents, video, audio, images

Extracted Visual Characteristics Include:

- Layout information (page structure, reading order)
- Figure detection and analysis
- Location metadata (page position, bounding box coordinates)
- Image content descriptions

Integration with RAG:

Content Understanding is optimized for retrieval-augmented generation scenarios, preserving the relationship between images and surrounding text.

3.2.6 Implement Video Analysis Workflows to Process and Interpret Video Segments

Definition: Processing video files to extract meaningful insights including visual frames, audio transcripts, and structured summaries.

Video Analysis Capabilities:

Capability	Description
Frame extraction	Extract key frames at defined intervals
Audio transcription	Transcribe spoken content with speaker diarization
Temporal alignment	Align visual content with audio timing
Segment analysis	Process video in logical segments (scenes, shots)
Summarization	Generate structured summaries of video content

Service Comparison:

Use Azure Video Indexer	Use Azure OpenAI	Use Azure Content Understanding
Advanced video analysis, transcription, translation of content	Broad, non-specific video understanding	Structured extraction with defined outputs
Face detection/redaction	Conversational Q&A about video	RAG-optimized video content
Multi-language support	Quick summarization	Temporal alignment

Implementation with Content Understanding:

python

```
# Conceptual - Using prebuilt-videoSearch analyzer
result = content_understanding.analyze(
    file="meeting_recording.mp4",
    analyzer="prebuilt-videoSearch"
)
# Returns: transcript phrases, timing info, key frame references, segments
```

3.2.7 Configure Single-Task and Pro-Mode Content Understanding Pipelines

Definition: Content Understanding offers different operational modes for various extraction scenarios.

Pipeline Modes:

Mode	Description	Best For
Single-Task	Process one file with one analyzer, get immediate results	Ad-hoc analysis, testing, low-volume scenarios
Pro-Mode	Batch processing with multiple analyzers, workflow integration	Production-scale, automated pipelines, scheduled jobs

Pro-Mode Features:

- Integration with Azure AI Search as a skill
- Support for chunking (breaking content into manageable pieces)
- Extraction options (images, location metadata)
- Can be combined with indexers for automated ingestion

Configuration Example (Skill Definition):

```
json
{
  "extractionOptions": ["images", "locationMetadata"],
  "chunkingProperties": {
    "unit": "characters",
    "maximumLength": 1325,
```

```
  "overlapLength": 0
 }
}
```

3.2.8 Implement Solutions That Identify Objects, Components, or Regions Within Images or Video

Definition: Detecting and localizing specific visual elements within images or video frames.

Primary Service:

- **Azure Vision Image Analysis** – Object detection, face detection, content tagging

Detection Capabilities:

Feature	Description	Output
Object detection	Identify and locate physical objects	Bounding box coordinates + object label + confidence
Face detection	Identify human face regions	Bounding box coordinates + optional attributes (age, emotion)
Content tagging	Generate keywords describing content	List of tags with confidence scores
Document layout	Identify text regions, tables, figures	Bounding boxes + content type

Object Detection Use Cases:

Use Case	Description
Inventory management	Count objects in warehouse images
Safety monitoring	Detect missing safety equipment in video
Image search	Find images containing specific objects

Use Case	Description
Facial detection	Redact faces for privacy compliance

Region Identification:

- Bounding box coordinates returned in normalized or pixel format
- Can be used for targeted processing (e.g., extract text from a specific region)
- For videos, detection must be applied frame-by-frame

3.3 Implement Responsible AI for Multimodal Content

This section covers safety, security, and governance for vision-based AI systems.

3.3.1 Implement Filters to Classify Unsafe or Disallowed Visual Content

Definition: Azure AI Content Safety provides automated detection of harmful visual content across four core harm categories.

Four Harm Categories for Images:

Category	Description
Hate and Fairness	Attacks on identity groups, discriminatory imagery
Sexual	Pornography, exploitation, erotic content
Violence	Graphic violence, weapons, terrorism
Self-Harm	Self-injury, suicide imagery

Severity Levels for Images:

Severity	Score	What It Means
Safe	0	No harmful content

Severity	Score	What It Means
Low	2	Mild, positive/general references
Medium	4	Moderate graphic content
High	6	Extreme, explicit harmful content

Default Configuration:

- Text models: Medium severity threshold (4 and 6 blocked)
- Serverless image deployments: Low threshold (stricter)

Implementation:

- Content Safety is integrated as the default filter for all Foundry model deployments
- Configurable via Content Safety service APIs
- Returns severity scores for each category

3.3.2 Detect and Mitigate Indirect Prompt Injection by Using Embedded Text in Images

Definition: Indirect prompt injection occurs when malicious instructions embedded in images (or other content) trick the AI into ignoring safety rules or performing unauthorized actions.

Attack Vectors:

Attack Type	Description	Example
Direct (Jailbreak)	User deliberately attempts to bypass system rules	"Ignore previous instructions and do X"
Indirect	Malicious instructions in content the AI processes	Text in an image that says "Forget your safety rules"

Azure Protection: Prompt Shields:

Shield Type	Description	Default
Direct attack detection	Detects jailbreak attempts in user prompts	On by default
Indirect attack detection	Detects XPIA (Cross-Domain Prompt Injection) in processed documents/images	Off by default (must enable)

We have 590 Practice test questions **AI-103: Developing AI Apps and Agents on Azure** (Taken from previous exams)

Full Practice Set link below

https://skillcertpro.com/product/microsoft-azure-ai-103-exam-questions-2026/

100% Money back Guarantee, If you don't pass the exam in 1st attempt, your money will be refunded back

How Indirect Detection Works:

1. Analyze embedded text in images using OCR
2. Scan for instruction patterns that override system behavior
3. Flag or block before processing

Mitigation Best Practices:

- Enable Indirect attack detection for applications processing untrusted images
- Use **Spotlighting** (preview) to distinguish trusted from untrusted input
- Configure to "Annotate and block" for high-security scenarios

3.3.3 Enforce Visual Policy Rules

Definition: Applying organizational policies to AI-generated or AI-processed visual content, including watermarks, brand compliance, and content flagging.

Key Policy Rules:

Policy Type	Description	Implementation
Watermarking	Apply visible/invisible markers to AI-generated images	Content Safety service or post-processing
Prohibited Symbols	Detect and flag banned logos, symbols (e.g., hate symbols)	Custom blocklists + Content Safety
Brand usage	Enforce brand guidelines (colors, logo placement)	Custom vision models or Azure Vision analysis
Inappropriate content	Flag NSFW or policy-violating imagery	Content Safety harm categories

Watermarking Use Cases:

- Identifying AI-generated content (transparency)
- Copyright protection
- Content provenance tracking

Implementing Policy Rules:

1. **Content Safety:** Configure severity thresholds and blocklists
2. **Azure Vision:** Analyze images for specific objects/symbols
3. **Custom processing:** Post-generation checks before returning to user

Example Brand Compliance Workflow:

Generate image → Detect logo placement → Verify colors match brand guide → If violation, reject or trigger manual review → Apply watermark → Return

Domain 4: Implement text analysis solutions (10–15%)

This domain covers text-based AI solutions and speech integration, representing 10-15% of the exam. It focuses on applying language models (LLMs) for text analysis, document processing, translation, and integrating speech as a conversational modality.

4.1 Apply Language Model Text Analysis

This section focuses on using generative AI to extract structured insights from unstructured text.

4.1.1 Implement Solutions to Extract Entities, Topics, Summaries, and Structured JSON Outputs

Definition: Using generative LLMs (instead of traditional NLP) to identify key information pieces from text and format them as structured data.

Key Extraction Capabilities:

Capability	Description	Example
Entity Extraction	Identify people, organizations, dates, locations, product names	"Microsoft was founded by Bill Gates" → {company: "Microsoft", person: "Bill Gates"}
Topic Extraction	Identify main subjects or themes in content	News article → {topics: ["economy", "technology"]}
Summarization	Generate concise abstracts or key point summaries	10-page report → 1-paragraph summary
Structured Outputs (JSON)	Enforce schema-constrained output for programmatic use	Invoice text → {invoice_number: "INV-001", total: 1250.00}

Implementation Approaches:

Method	Best For	Example
Prompt Engineering	Ad-hoc, variable extraction tasks	"Extract all names and dates as JSON"
Content Understanding Analyzers	Reusable, domain-specific extraction	prebuilt-invoice, prebuilt-idDocument
Output Schema Enforcement	Forcing specific JSON structure	JSON Schema validation, Structured Outputs API

Content Understanding Analyzer Types for Text:

- **Document base analyzers:** Process text, layout, tables, and embedded images
- **Domain-specific analyzers:** Pre-built for invoices, receipts, ID documents, contracts
- **Custom analyzers:** Extend base analyzers with custom field definitions

Analyzer Configuration Example:

```
json
{
  "analyzerId": "customInvoiceAnalyzer",
  "baseAnalyzerId": "prebuilt-document",
  "fieldSchema": {
    "fields": {
      "vendorName": { "type": "string" },
      "invoiceTotal": { "type": "number" }
    }
  }
}
```

Output Options:

- **Markdown**: Human-readable formatted output (default annotationFormat: "markdown")
- **JSON**: For programmatic consumption
- **Confidence scores**: Enable estimateFieldSourceAndConfidence for extraction reliability metrics

Key Takeaway: Use **Content Understanding analyzers** for reusable, domain-specific extraction. Use **direct prompting** with JSON schema enforcement for ad-hoc tasks.

4.1.2 Configure Detection of Sentiment, Tone, Safety Issues, and Sensitive Content

Definition: Implementing automated detection of emotional content, risky language, and policy violations in user inputs or model outputs.

Detection Categories:

Category	Description	Severity Levels
Sentiment/Tone	Positive, negative, neutral, mixed; angry, frustrated, satisfied	Not severity-scored (categorical)
Safety - Hate & Fairness	Discriminatory content, attacks on identity groups	Low / Medium / High
Safety - Violence	Threats, graphic violence, terrorist content	Low / Medium / High
Safety - Sexual	Sexually explicit content, exploitation	Low / Medium / High
Safety - Self-Harm	Self-injury, suicide promotion	Low / Medium / High
Sensitive Content	PII (personally identifiable information), secrets, credentials	Binary (present/absent)

Azure Services for Detection:

Service	Primary Use
Azure AI Content Safety	Text and image safety moderation (hate, sexual, violence, self-harm)
Azure AI Language (Sentiment)	Sentiment and opinion mining
Azure OpenAI (with safety system prompt)	Custom tone/sentiment with safety filters

Content Safety Integration:

- Content Safety is the **default filter** for all Foundry model deployments
- Blocks content based on harm category and severity thresholds
- Default: Medium severity (4 and 6 on 0-6 scale) blocked

Content Safety Configuration:

Configure thresholds per harm category:

- Hate/Fairness: Low / Medium / High
- Sexual: Low / Medium / High
- Violence: Low / Medium / High
- Self-Harm: Low / Medium / High

Temporary Blocks ("403 Content Violation"):

The Azure OpenAI service may apply **temporary resource-level blocks** based on aggregated safety signals, evaluating both content and usage patterns (volume, repetition, automation).

Key Takeaway: Content Safety is **on by default** for all deployments. Configure severity thresholds per harm category. Monitor for 403 errors indicating aggregated safety enforcement.

4.1.3 Build Solutions That Translate Text

Definition: Implementing language translation using either Azure's neural machine translation (NMT) or LLMs for higher quality results.

Translation Options in Foundry:

Option	Model Choice	Best For
Azure Translator (NMT)	Neural Machine Translation	Standard translation, 100+ languages, cost-efficient
Azure Translator (LLM-powered)	GPT-4o, GPT-4o mini	Gender/tone awareness, domain customization
LLM Direct Translation	Any LLM via chat completions	Context-aware, high-quality, smaller volumes

Configuration Options:

- **Gender customization:** Specify masculine/feminine/neutral output
- **Tone customization:** Formal, casual, professional

- **Domain data:** Refine results using your terminology

Document Translation:

- Batch API supports PDF, Office docs, images, scanned PDFs
- **IMPORTANT:** LLM translation is NOT yet available in document translation API (only via translate text API)
- For LLM-powered document translation, use RAG pattern: extract text → translate with LLM

Code Example (Text Translation with Translator):

```
python
# Conceptual - Azure Translator API
response = translator.translate(
    text="Hello, how are you?",
    to_language="es",
    model="gpt-4o",  # LLM option (preview)
    gender="neutral",
    tone="formal"
)
```

Key Takeaway: Use **Azure Translator with LLMs** (GPT-4o) for gender/tone customization. **LLM translation is currently text-only**, not available for batch document translation.

We have 590 Practice test questions **AI-103: Developing AI Apps and Agents on Azure** (Taken from previous exams)

Full Practice Set link below

https://skillcertpro.com/product/microsoft-azure-ai-103-exam-questions-2026/

100% Money back Guarantee, If you don't pass the exam in 1st attempt, your money will be refunded back

4.1.4 Customize Language Model Outputs for Domain Tasks

Definition: Tailoring LLM outputs for specific industry contexts such as regulatory compliance, legal review, or healthcare summarization.

Customization Approaches:

Approach	Method	When to Use
Fine-tuning	Train model on domain-specific data (compliance docs, legal briefs)	High-volume, specialized tasks with consistent patterns
Custom Analyzers	Content Understanding with domain-specific field schemas	Structured extraction with reusable configurations
Prompt Engineering	Detailed system prompts with domain rules and examples	Rapid iteration, smaller volumes
Agentic Workflows	Multi-step processing with gap analysis rules	Complex, multi-document domain workflows

Domain Extraction with Content Understanding:

Extend prebuilt-document analyzer with custom fields for domain-specific extraction:

```
json
{
  "baseAnalyzerId": "prebuilt-document",
  "fieldSchema": {
    "fields": {
      "complianceRegulation": { "type": "string" },
      "riskLevel": { "type": "string", "enum": ["Low", "Medium", "High"] },
      "actionRequiredBy": { "type": "date" }
    }
  }
}
```

Example Workflow: Compliance Summarization

1. Extract required fields using custom analyzer with confidence scoring
2. Generate AI-powered summary across document set
3. Run gap analysis using YAML-based ruleset

Key Takeaway: Use **custom fields** in Content Understanding analyzers for reusable domain extraction. For agentic domain workflows, leverage the **Agent Framework Workflow Engine** with gap analysis rules.

4.2 Implement Speech Solutions

This section covers voice-enabled AI, including speech-to-text, text-to-speech, and multimodal speech reasoning.

4.2.1 Implement Workflows to Convert Speech to Text and Text to Speech for Agentic Interactions

Definition: Enabling voice-based conversations with AI agents by converting spoken input to text (STT) and generating spoken responses (TTS).

Azure Speech Services:

Service	Purpose
Speech-to-Text (STT)	Convert real-time/batch audio to text
Text-to-Speech (TTS)	Convert text to natural-sounding speech
Custom Speech	Fine-tune STT for domain vocabulary
Custom Neural Voice	Train brand-specific voices

Voice Agent Architecture Patterns:

From maximum control to fully managed:

Pattern	Description	STT	TTS
Cascade (Max Control)	You manage each component independently	Azure Speech, Whisper, Custom Speech	Azure Neural Voices, Custom Neural Voice
VoiceLive (Managed Pipeline)	Azure manages pipeline; you configure models	Azure Speech (15 locales) + phrase lists	Neural Voices + HD voices
Direct Realtime (Minimal)	Single API handles all	OpenAI built-in	OpenAI voices

Component Options in Cascade Pattern:

- **STT:** Azure Speech, Whisper (via Azure OpenAI), or custom fine-tuned models
- **LLM:** Any Foundry model, swappable without code changes
- **TTS:** Azure Neural Voices (multilingual, HD, Custom Neural Voice)

Production Considerations:

- **Barge-in:** User interrupts agent mid-response; requires VAD (Voice Activity Detection)
- **Noise suppression + echo cancellation:** Available in VoiceLive pattern
- **Telephony integration:** Azure Communication Services (ACS) for PSTN calling

Key Takeaway: Use **Cascade** for on-premises or component flexibility. Use **VoiceLive** for managed infrastructure with STT/TTS customization. Both support phrase lists, custom lexicon, and custom neural voices.

4.2.2 Integrate Speech as an Agent Modality, Including Custom Speech Models

Definition: Enabling voice as a bidirectional communication channel for agents, with custom models for domain-specific recognition.

Custom Speech Models:

- Train on your domain's vocabulary (medical terms, product names, acronyms)

- Improves recognition accuracy for specialized content

Customization Type	Description
Phrase Lists	Just-in-time vocabulary hints for short-term accuracy
Custom Speech Model	Fine-tuned STT models trained on your domain data
Custom Lexicon	Pronunciation customization for unusual terms
Custom Neural Voice	Brand voice trained on your audio samples

Speech Configuration Options:

- **Language options:** Automatic multilingual (15 locales), single language, or up to 10 defined languages
- **Phrase lists:** Provide product names, acronyms, industry terms at request time

Enterprise Speech Integration:

Both Cascade and VoiceLive support:

- Custom Speech (STT fine-tuning)
- Custom Neural Voice (brand voice)
- Phrase lists and custom lexicon

Key Takeaway: Custom Speech models improve accuracy for domain-specific vocabulary. Supported in both Cascade and VoiceLearn architecture patterns.

4.2.3 Enable Multimodal Reasoning from Audio Inputs

Definition: Using multimodal models that can process audio inputs alongside text and images for comprehensive understanding.

Audio-Enabled Multimodal Models:

Some Foundry models can accept audio input directly and reason across modalities.

Code Example - Audio Processing with Multimodal Model:

```
python
from azure.ai.inference.models import (
    TextContentItem, AudioContentItem, InputAudio,
    AudioContentFormat, SystemMessage, UserMessage
)

response = client.complete(
    messages=[
        SystemMessage("You are an AI assistant for translating audio clips."),
        UserMessage([
            TextContentItem(text="Translate this audio to Spanish."),
            AudioContentItem(
                input_audio=InputAudio.load(
                    audio_file="audio.mp3",
                    audio_format=AudioContentFormat.MP3
                )
            )
        ])
    ]
)
```

Audio Input Methods:

1. **Local file:** Base64-encoded audio in payload
2. **Cloud URL:** Using audio_url type with publicly accessible file

Audio Token Usage:

- Audio is tokenized and billed based on processing time/pricing model
- Models may use internal STT modules → different token counting strategies
- **Check model cards** for specific behavior

Supported Audio Formats:

- MP3, WAV, FLAC, OGG (format-specific support varies by model)

Use Cases:

- Audio transcription + translation in single call
- Customer service call analysis
- Multilingual meeting summarization

Key Takeaway: Use AudioContentItem to pass audio directly to multimodal models for transcription, translation, or combined reasoning without separate STT preprocessing.

4.2.4 Translate Speech into Other Languages

Definition: Converting spoken content from one language to another, using either two-step (STT + translation) or direct speech translation.

Approaches:

Approach	Method	Latency	Quality
Two-step (STT → Translation)	Speech → Text (STT) → Translate text	Higher (two passes)	High with LLM translation
Direct Multimodal	Audio input → Multimodal model → Text output in target language	Lower (single pass)	Depends on model

Two-step Implementation:

User Speech (language A)

→ Azure Speech-to-Text

→ Text in Language A

→ Translator / LLM

→ Text in Language B

→ (Optional) Text-to-Speech

→ Spoken output (language B)

Direct Multimodal Implementation:

```
python
# Single API call: audio in Spanish → English text translation
response = multimodal_client.complete(
  messages=[
    SystemMessage("Translate the spoken audio to English."),
    UserMessage([AudioContentItem(
      input_audio=InputAudio.load("spanish_audio.mp3", AudioContentFormat.MP3)
    )])
  ]
)
# Returns English translation directly
```

When to Use Each:

- **Two-step:** When you need intermediate text for other processing, or when using specialized STT for challenging audio
- **Direct:** Lowest latency, when translation is the only goal

Key Takeaway: Multimodal models can translate speech directly in one API call. Two-step (STT → Translator) allows LLM-powered translation with gender/tone customization.

Domain 5: Implement information extraction solutions (10–15%)

This domain focuses on the end-to-end pipeline for preparing content for AI consumption, covering **retrieval and grounding pipelines** followed by **document content extraction**. It accounts for 10-15% of the exam and is critical for understanding how to feed high-quality, relevant data to generative AI models.

5.1 Build Retrieval and Grounding Pipelines

This section covers ingesting, enriching, and indexing multimodal content so it can be retrieved by LLMs to reduce hallucinations (Grounding).

5.1.1 Ingest and Index Content, Such as Documents, Images, Audio, and Video

Definition: The process of pulling raw data from a source (e.g., Blob Storage, Cosmos DB), cracking it to extract text, and loading it into a searchable index.

Workflow Components:

1. **Data Source:** Azure Blob Storage, Data Lake, or SharePoint.
2. **Indexer:** The crawler that connects to the data source and initiates the pipeline.
3. **Document Cracking:** The process where the indexer extracts text and images from binary files (PDFs, Office docs).

Supported Formats:

- **Documents:** PDF (up to 2,000 pages for paid tier), DOCX, XLSX, PPTX, HTML.
- **Images:** JPEG, PNG, BMP, TIFF.

Practical Tip: To index images, you *must* enable **OCR (Optical Character Recognition)** during the indexing process to convert visual text into machine-readable text.

We have 590 Practice test questions **AI-103: Developing AI Apps and Agents on Azure** (Taken from previous exams)

Full Practice Set link below

https://skillcertpro.com/product/microsoft-azure-ai-103-exam-questions-2026/

100% Money back Guarantee, If you don't pass the exam in 1st attempt, your money will be refunded back

5.1.2 Configure Semantic Search, Hybrid Search, and Vector Search for Grounding

Definition: Different search methods used to retrieve the most relevant chunks of data to provide context to an LLM.

Comparison Chart:

Search Type	How it Works	Key Technique	Best For
Keyword (Lexical)	Matches exact words or phrases.	BM25 Ranking	Specific error codes, names, or legal jargon.
Vector	Converts text to numbers (embeddings). Finds *meaning* rather than exact words.	HNSW (Hierarchical Navigable Small World)	Conceptual questions (e.g., "How do I fix connectivity?" matching with network setup docs).
Hybrid	**Parallel execution** of Keyword + Vector. Results are merged and re-ranked.	RRF (Reciprocal Rank Fusion)	**Default for RAG:** balances precision and recall.

Configuration Details:

- **Hybrid Request Structure:**

```
json
{
  "search": "text query here",      // Keyword part
  "vectorQueries": [              // Vector part
    { "vector": [0.01, ...], "fields": "contentVector", "k": 50 }
  ],
  "hybridSearch": {
    "maxTextRecallSize": 1000      // Limits keyword results for RRF
  }
}
```

- **Performance Tuning:** To prevent throttling (429 errors), keep k values between 30-50 and avoid enabling Semantic ranking unless required.

Key Takeaway: Hybrid search is the standard for Azure AI Search RAG implementations. It uses **Reciprocal Rank Fusion (RRF)** to combine results.

5.1.3 Implement Enrichment by Using Custom or Built-in Skills

Definition: Using AI Skills (built-in or custom) to add metadata or extract information from raw data before it is indexed.

Built-in Skills (Azure AI Services):

- **Vision (Image Analysis):** OCR, generate captions/descriptions of images.
- **Language (Text Analytics):** Translate, Entity Recognition (People/Places), Key Phrase Extraction.
- **Document Intelligence (Layout):** Extract structure (tables, headings) and output as **Markdown**.

Custom Skills:

- **What:** Custom code (usually Azure Functions) that processes specific data.
- **Use Case:** Applying a specific company algorithm or cleaning proprietary data formats.

Skill Definition (Document Layout Skill):

```
json
{
 "@odata.type": "#Microsoft.Skills.Util.DocumentIntelligenceLayoutSkill",
 "context": "/document",
 "outputFormat": "markdown",   // Converts PDF/Word to LLM-friendly Markdown
 "extractionOptions": ["images", "locationMetadata"], // Extract images and page numbers
 "inputs": [ { "name": "file_data", "source": "/document/file_data" } ],
 "outputs": [ { "name": "markdown_document" } ]
}
```

Note: This skill uses the Document Intelligence Layout model to preserve reading order and headings.

5.1.4 Configure RAG Ingestion Flow, Including Documents and Using OCR

Definition: The specific pipeline steps required to prepare multimodal documents for Retrieval-Augmented Generation.

The Ingestion Pipeline:

1. **Cracking:** Indexer reads the file.
2. **OCR (Optical Character Recognition):** Extracts text from scanned images inside PDFs or standalone picture files. *Enabled by the Layout Skill* (parameter: enableOcr: true).
3. **Enrichment:** (Optional) Apply skills (e.g., Image tagging).
4. **Chunking (Semantic Splitting):**
    - **Goal:** Break large documents into smaller pieces.
    - **Best Practice:** Use **Semantic Chunking** (splitting based on headings/tables) via Markdown output, rather than fixed-size chunks. This maintains context, especially for tables cited by the LLM.
5. **Vectorization:** Convert the text chunk into vectors (using an Embedding model like text-embedding-ada-002) and store them in the index.

5.1.5 Connect Retrieval Pipelines Directly to Workflows and Agent Tools

Definition: Exposing the indexed search results to Agents so they can "Ground" their responses.

Implementation: Agent Knowledge Bases (MCP):

Foundry Agents can connect to Azure AI Search via the **Model Context Protocol (MCP)**.

- **The Connection:** A RemoteTool connection is created in the Foundry project, pointing to the Search endpoint.
- **Agent Behavior:** The agent calls the knowledge base to retrieve data. It uses **Hybrid + Semantic ranking** for retrieval.
- **Citation Requirement:** Agent instructions **must** be configured to force the agent to cite sources. If the answer isn't in the search results, the agent must say "I don't know" to prevent hallucinations.

Example Agent Tool Code:

python

```
mcp_kb_tool = MCPTool(
```

```
    server_url="{search_endpoint}/knowledgebases/{kb_name}/mcp",
    allowed_tools=["knowledge_base_retrieve"],
    require_approval="never" # For auto mode
)
```

Key Takeaway: MCP is the standard protocol for connecting agents to Search indexes. Agents should use *extractive* mode (verbatim content retrieval) rather than synthetic answers to maintain control over the LLM response.

5.2 Extract Content from Documents

This section focuses on **Content Understanding** in Foundry Tools—how to configure analyzers to extract structured fields and clean text.

5.2.1 Extract Information by Using Multimodal Pipelines

Definition: Combining OCR, layout analysis, and Language models to understand complex documents containing text, tables, and images.

The Analyzer Concept:

The primary unit of extraction is the **Analyzer**. It defines *what* to process and *how* to output it.

Configuration Options:

- enableOcr **(Default: true):** Extracts text from scanned pages.
- enableLayout **(Default: true):** Detects paragraphs, headings, and reading order.
- enableFigureDescription **(Default: false):** Generates Alt-text for images/diagrams (great for accessibility).
- tableFormat**:** Specify "html" (for complex) or "markdown" (for LLM ingestion).

5.2.2 Produce Clean, Grounded Representations to Use with Agents and RAG

Definition: Specifically configuring analyzers to output data in formats optimized for LLMs (Markdown) rather than raw JSON.

Output Formats:

- **Markdown (**annotationFormat: "markdown"**):** Preserves headings (#), lists, and tables. This is the **preferred format** for RAG because LLMs understand markdown structure naturally.
- **Text (**outputFormat: "text"**):** Raw text, often used with chunking.

Chunking Properties for RAG:

```
json
"chunkingProperties": {
   "unit": "characters",
   "maximumLength": 1325,     // Size of each chunk
   "overlapLength": 200       // Context overlap between chunks
}
```

5.2.3 Implement Analyzers for Generating Structured or Markdown Outputs

Definition: Creating reusable configurations (Analyzers) to extract custom fields (like specific contract expiry dates) or standard ones.

Types of Analyzers:

1. **Base Analyzers:** prebuilt-document, prebuilt-image, prebuilt-video. Use as foundation.
2. **Domain-Specific Analyzers:** prebuilt-invoice (extracts total, tax, vendor), prebuilt-idDocument, prebuilt-contract.
3. **Custom Analyzers (Exam Critical):** Extending a Base Analyzer to extract specific fields.

Custom Analyzer Definition:

```
json
{
  "analyzerId": "my-contract-analyzer",
  "baseAnalyzerId": "prebuilt-document",   // Start with document layout
  "config": { "enableOcr": true },
  "fieldSchema": {                  // Where you define custom extraction
    "fields": {
```

```
    "ContractEndDate": { "type": "date" },
    "TotalValue": { "type": "number" }
  }
 }
}
```

Exam Tip: You must define a fieldSchema to get structured JSON output. Without it, you only get Markdown/text.

We have 590 Practice test questions **AI-103: Developing AI Apps and Agents on Azure** (Taken from previous exams)

Full Practice Set link below

https://skillcertpro.com/product/microsoft-azure-ai-103-exam-questions-2026/

100% Money back Guarantee, If you don't pass the exam in 1st attempt, your money will be refunded back

Disclaimer: All data and information provided on this site is for informational purposes only. This site makes no representations as to accuracy, completeness, correctness, suitability, or validity of any information on this site & will not be liable for any errors, omissions, or delays in this information or any losses, injuries, or damages arising from its display or use. All information is provided on an as-is basis.